Engineering Standards

Part 1: Engineering Principles & Architecture Rules

Bidra.no Platform

Version 1.0

---

1. Purpose

This document defines the engineering principles, architectural rules and software development standards for the Bidra platform.

These standards are mandatory.

Every engineer, contractor, AI coding assistant, automation tool and future development team must follow these rules.

The objective is to ensure that Bidra remains:

- Maintainable
- Scalable
- Secure
- Testable
- Predictable
- Consistent
- Extensible

---

2. Engineering Philosophy

Bidra is not a collection of pages.

It is a long-term digital platform.

Every engineering decision should optimize for:

- readability
- correctness
- maintainability
- simplicity
- scalability

Never optimize for writing less code.

Optimize for writing code that someone else can safely understand two years later.

---

3. Core Principles

The following principles are mandatory.

3.1 Simplicity First

Prefer simple solutions.

Avoid unnecessary abstraction.

Avoid premature optimization.

Avoid unnecessary patterns.

Simple code always wins.

3.2 Explicit Over Implicit

Business logic should always be obvious.

Bad:

```
calculate()
```

Good:

```
calculatePlatformFee()
```

Every function should clearly communicate its purpose.

3.3 Readability Over Cleverness

Never write clever code.

Write boring code.

Future developers should understand the implementation immediately.

3.4 Business Rules Live in Code

Business rules must never be hidden inside:

- controllers
- React components
- SQL queries
- API routes
- Prisma calls

Business rules belong inside domain services.

3.5 Single Source of Truth

Every piece of information must have one authoritative owner.

Examples:

Financial balances

→ Financial Ledger

Time Credit balance

→ Time Credit Ledger

Campaign progress

→ Progress Events

Never calculate critical values differently in different modules.

---

4. Architectural Style

Bidra shall use:

Modular Monolith Architecture

The application behaves as one deployment while being internally divided into independent business modules.

Each module owns:

- business logic
- entities
- repositories
- services
- events
- permissions

Modules communicate through services and domain events.

Never through direct database access.

---

5. Domain-Driven Design (DDD)

The platform should follow Domain-Driven Design principles.

Each business capability is an independent domain.

Examples:

- Authentication
- Organizations
- Campaigns
- Needs
- Contributions
- Payments
- Volunteers
- Time Credits
- Rewards
- Notifications
- Search
- Administration

Each domain owns its own business logic.

---

6. Module Independence

Modules must never bypass each other.

Wrong:

```
Campaign Module

↓

queries Payment database tables directly
```

Correct:

```
Campaign Module

↓

Payment Service

↓

Payment Repository
```

Only the owning module may directly access its database tables.

---

7. Separation of Responsibilities

Every layer has one responsibility.

Controller

Responsible for:

- HTTP requests
- validation
- authentication
- authorization
- response formatting

Controllers must never contain business logic.

Service

Responsible for:

- business rules
- orchestration
- workflows
- domain validation

Services coordinate work.

Repository

Responsible only for persistence.

Repositories:

- query database
- insert
- update
- delete

Repositories must not contain business rules.

Entity

Represents business objects.

Entities should express business concepts rather than database implementation details.

Worker

Responsible for asynchronous processing.

Examples:

- email
- Stripe webhook processing
- notifications
- QR generation
- report generation

---

8. SOLID Principles

The platform should follow SOLID.

Single Responsibility

One class.

One purpose.

Open / Closed

Extend behavior.

Do not modify stable implementations unnecessarily.

Liskov Substitution

Implementations must remain interchangeable.

Interface Segregation

Small focused interfaces.

Avoid giant interfaces.

Dependency Inversion

Depend on abstractions.

Not implementations.

---

9. Dependency Rules

Dependencies always point inward.

Example:

```
Controller

↓

Service

↓

Repository

↓

Database
```

Never:

```
Repository

↓

Controller
```

---

10. Forbidden Dependencies

The following are prohibited.

Campaign module importing Payment repository.

Volunteer module updating Payment tables.

Need module modifying User tables.

Frontend importing database code.

Controller importing Prisma directly.

React components making SQL assumptions.

Modules sharing mutable state.

---

11. Communication Between Modules

Preferred communication order:

Direct Service

↓

Domain Event

↓

Background Job

↓

External API

Example:

Contribution Verified

↓

Contribution Service

↓

Need Service

↓

Need Progress Updated

↓

Emit Event

↓

Notification Worker

---

12. Domain Events

Every important business event should become a domain event.

Examples:

```
UserRegistered

OrganizationCreated

CampaignPublished

NeedCompleted

ContributionVerified

PaymentSucceeded

VolunteerAttendanceVerified

TimeCreditsAwarded

RewardRedeemed
```

Events should describe facts.

Not commands.

Good:

PaymentSucceeded

Bad:

UpdateNeedProgress

---

13. CQRS Guidance

Bidra is not full CQRS.

Instead:

Simple CRUD

→ standard services

Complex reporting

→ read models

Analytics

→ optimized queries

Do not introduce CQRS unless complexity requires it.

---

14. Transaction Boundaries

Every business transaction should be atomic.

Examples:

Donation succeeds:

- Payment created
- Ledger updated
- Contribution verified
- Need updated

All succeed.

Or none succeed.

Use database transactions.

---

15. Immutable Data

Certain data must never change.

Examples:

Financial Ledger

Time Credit Ledger

Audit Logs

Payment History

Progress Events

Corrections create new entries.

Never overwrite history.

---

16. Public IDs vs Internal IDs

Every object has two identifiers.

Internal

`UUID`

Public

`#12345`

Rules:

UUID

Never exposed publicly.

Public IDs

Used in URLs

QR codes

Customer support

Search

---

17. Error Handling Philosophy

Errors are expected.

Every error should be:

- explicit
- typed
- logged
- user friendly

Never expose:

- SQL errors
- stack traces
- secrets
- internal implementation

---

18. Logging Standards

Every important operation should produce structured logs.

Every log contains:

- timestamp
- request ID
- user ID
- organization ID
- module
- action
- duration
- status

Never log:

- passwords
- tokens
- card numbers

- secrets

---

19. Audit Philosophy

Audit logs are not application logs.

Audit logs represent:

who

did what

to which object

when

why

Audit logs are immutable.

---

20. Security by Default

Every feature should assume hostile input.

Every endpoint must validate:

- permissions
- ownership
- object state
- input

Trust nothing from the client.

---

21. Performance Philosophy

Optimize only after measuring.

Never optimize based on assumptions.

Measure:

- API latency
- SQL performance
- memory
- queue times
- rendering

---

22. Database Philosophy

The database is the source of truth.

Never trust cached data.

Caches improve speed.

Databases provide correctness.

---

23. API Philosophy

REST should represent business concepts.

Good:

```
POST /campaigns
```

```
POST /needs
```

```
POST /contributions
```

Bad:

```
POST /saveCampaign
```

```
POST /updateNeedProgressNow
```

Resources represent nouns.

HTTP methods represent actions.

---

24. Frontend Philosophy

React components should display information.

Business logic belongs on the server.

The frontend should never determine:

- platform fees
- permissions
- Time Credit awards
- ledger calculations
- financial balances

---

25. AI Engineering Rules

Every AI coding tool must follow these principles.

AI must:

- preserve architecture
- follow module boundaries
- use existing services
- reuse components
- write tests
- update documentation
- respect naming conventions

AI must never:

- bypass services
- duplicate business logic
- introduce hidden dependencies
- ignore validation
- hardcode configuration
- change database schema without migration
- modify ledger history
- expose internal UUIDs
- disable audit logging

Generated code should always look as though it was written by an experienced engineer following the project's conventions.

---

26. Refactoring Principles

Refactoring should:

- improve readability

- reduce duplication
- preserve behavior
- maintain tests

Refactoring must never introduce functional changes unless explicitly approved.

---

27. Technical Debt Policy

Technical debt is acceptable only when:

- documented
- prioritized
- temporary
- tracked

Every shortcut must include:

- reason
- impact
- planned resolution

Undocumented technical debt is not acceptable.

---

28. Architecture Decision Records (ADR)

Significant architectural decisions must be documented as ADRs.

Each ADR should include:

- Decision title
- Date
- Status
- Context
- Options considered
- Decision
- Consequences

Examples requiring ADRs:

- Replacing PostgreSQL
- Introducing microservices
- Changing authentication strategy
- Replacing Stripe
- Changing search engine

---

29. Definition of Engineering Excellence

Engineering work is considered complete only when:

- The solution follows the architecture.
- Business logic is placed in the correct layer.
- Code is readable and self-explanatory.
- Tests pass.
- Security requirements are met.
- Performance requirements are met.
- Documentation is updated.
- Audit logging is preserved.
- No duplicated business logic exists.
- Public APIs remain consistent.

---

30. Engineering Constitution

The following rules are non-negotiable:

1. Controllers never contain business logic.
2. Repositories never contain business rules.
3. Services own business logic.
4. Modules never access another module's database directly.
5. Financial records are immutable.
6. Time Credit records are immutable.
7. Audit logs are immutable.
8. Public IDs are exposed, UUIDs are not.
9. Every sensitive action is audited.
10. Every business workflow is testable.
11. Configuration belongs in configuration, not in source code.
12. Security is enforced on the server.
13. Domain events describe completed business facts.
14. Code is written for maintainability before optimization.
15. Every engineer and every AI assistant follows these standards without exception.

Engineering Guiding Principle

When faced with multiple valid implementations, always choose the solution that is:

1. Easier to understand.
2. Easier to test.

3. Easier to maintain.
4. Easier to extend.
5. Least likely to introduce hidden coupling.

Bidra is intended to be a platform that evolves over many years. Every architectural decision should make future development easier, not harder.